

Cahier des Charges Technique

Plateforme de Gestion Automatisée des Quittances de Loyer

Version : 1.0

Date : 28 octobre 2025

Client : Cosynight SAS

Projet : Rental Receipt Platform

Durée estimée : 2 mois



Table des matières

1. [Contexte et objectifs](#)
2. [Périmètre fonctionnel](#)
3. [Spécifications techniques](#)
4. [Architecture système](#)
5. [Modèle de données](#)
6. [Interfaces utilisateur](#)
7. [Parcours utilisateurs](#)
8. [Exigences de sécurité](#)
9. [Performances et scalabilité](#)
10. [Contraintes et dépendances](#)
11. [Planning de réalisation](#)
12. [Critères de recette](#)
13. [Maintenance et évolutions](#)

1. Contexte et objectifs

1.1 Contexte métier

Cosynight est une société qui agit à la fois comme propriétaire et locataire dans le cadre de locations et sous-locations immobilières. L'entreprise doit gérer :

- **Des locataires** : Cosynight loue des biens à des particuliers et doit leur fournir des quittances de loyer
- **Des propriétaires** : Cosynight loue des biens auprès de propriétaires et doit leur envoyer des avis de paiement

Actuellement, cette gestion est manuelle, chronophage et sujette aux erreurs (oublis d'envoi, erreurs de montants, perte de documents).

1.2 Objectifs du projet

Objectif principal

Créer une plateforme web SaaS permettant de **générer, envoyer et archiver automatiquement** les quittances de loyer et avis de paiement.

Objectifs spécifiques

-  Réduire le temps de traitement administratif de 80%
-  Éliminer les erreurs de saisie manuelle
-  Garantir l'envoi systématique des documents à date
-  Centraliser l'archivage pour faciliter la comptabilité
-  Personnaliser les documents avec les informations de l'entreprise

1.3 Bénéfices attendus

Bénéfice	Impact
Gain de temps	2-3h/semaine économisées sur la gestion administrative
Fiabilité	100% des quittances envoyées à date, 0% d'oubli
Traçabilité	Historique complet et horodaté de tous les envois
Comptabilité	Export simplifié pour le cabinet comptable
Professionalisme	Documents personnalisés et conformes

2. Périmètre fonctionnel

2.1 Fonctionnalités principales (MVP)

2.1.1 Authentification et gestion des utilisateurs

US-001 : Connexion utilisateur

En tant qu'utilisateur,
Je veux me connecter avec mon email et mot de passe,
Afin d'accéder à la plateforme de manière sécurisée.

Critères d'acceptation :

- Formulaire avec email et mot de passe
- Message d'erreur si identifiants incorrects
- Redirection vers le dashboard après connexion réussie
- Session maintenue pendant 7 jours (remember me)
- Déconnexion possible à tout moment

US-002 : Gestion des rôles

En tant qu'administrateur,
Je veux que les accès soient restreints selon les rôles,
Afin de sécuriser les données sensibles.

Rôles :

- ADMIN : accès complet (locataires + propriétaires + paramètres)
- COMPTABLE : lecture seule sur tous les documents
- USER : création/envoi de quittances locataires uniquement

Critères d'acceptation :

- Les propriétaires ne sont visibles que par ADMIN
- Les paramètres entreprise modifiables uniquement par ADMIN
- Les COMPTABLE peuvent exporter mais pas modifier

2.1.2 Gestion des locataires

US-003 : Créer un locataire

En tant qu'utilisateur,
Je veux créer la fiche d'un nouveau locataire,
Afin de pouvoir lui générer des quittances.

Champs obligatoires :

- Prénom
- Nom
- Email
- Téléphone
- Adresse complète
- Montant du loyer (€)
- Date d'entrée dans le logement

Champs optionnels :

- Nom de société (si locataire professionnel)
- Redevance (montant à déduire du loyer)
- Motif de la redevance

Critères d'acceptation :

- Validation des champs en temps réel
- Email unique (pas de doublon)
- Format de téléphone français validé
- Montant du loyer > 0
- Date d'entrée ne peut être dans le futur

US-004 : Liste des locataires

En tant qu'utilisateur,
Je veux voir la liste de tous mes locataires actifs,
Afin de les gérer facilement.

Affichage :

- Tableau avec : Nom, Email, Loyer, Date d'entrée, Nb quittances
- Tri par nom/loyer/date
- Recherche par nom ou email
- Badge "actif" / "inactif"
- Actions : Voir détails / Modifier / Désactiver / Générer quittance

Critères d'acceptation :

- Liste chargée en < 2 secondes
- Pagination si > 50 locataires

- Recherche réactive (debounce 300ms)

US-005 : Modifier un locataire

En tant qu'utilisateur,
Je veux modifier les informations d'un locataire,
Afin de les maintenir à jour (changement de loyer, email, etc.).

Critères d'acceptation :

- Formulaire pré-rempli avec les données actuelles
- Validation identique à la création
- Message de confirmation après modification
- Historique des modifications non visible (v2)

US-006 : Désactiver un locataire

En tant qu'utilisateur,
Je veux désactiver un locataire qui a quitté le logement,
Afin de ne plus le voir dans la liste active tout en conservant son historique.

Critères d'acceptation :

- Soft delete (données conservées en base)
- Locataire n'apparaît plus dans la liste par défaut
- Historique de ses quittances reste accessible
- Possibilité de réactiver si erreur

2.1.3 Gestion des propriétaires

US-007 : Créer un propriétaire

En tant qu'administrateur,
Je veux créer la fiche d'un propriétaire,
Afin de lui envoyer des avis de paiement.

Champs obligatoires :

- Prénom
- Nom
- Email
- Téléphone

- Adresse complète
- Montant du loyer que Cosynight paie (€)

Champs optionnels :

- Nom de société
- Numéro SIRET

Critères d'acceptation :

- Validation identique aux locataires
- Email unique
- Accès réservé au rôle ADMIN

US-008 : Liste des propriétaires

En tant qu'administrateur,
Je veux voir la liste de tous les propriétaires,
Afin de gérer les biens loués.

Critères d'acceptation :

- Même structure que liste locataires
- Visible uniquement par rôle ADMIN
- Autres utilisateurs n'y ont pas accès (403)

2.1.4 Génération et envoi de quittances

US-009 : Générer une quittance pour un locataire

En tant qu'utilisateur,
Je veux générer une quittance de loyer en un clic,
Afin de l'envoyer automatiquement au locataire.

Processus :

1. Sélectionner un locataire
2. Sélectionner la période (mois/année)
3. Confirmer le montant (pré-rempli avec le loyer du locataire)
4. Prévisualiser le PDF
5. Envoyer par email

Critères d'acceptation :

- PDF généré selon le template Cosynight
- Email envoyé automatiquement avec le PDF en pièce jointe

- Confirmation visuelle de l'envoi
- Enregistrement dans l'historique
- Durée totale du processus < 10 secondes

US-010 : Prévisualisation du PDF

En tant qu'utilisateur,
Je veux prévisualiser le PDF avant envoi,
Afin de vérifier qu'il est correct.

Critères d'acceptation :

- Viewer PDF intégré dans la modal
- Boutons : "Télécharger" / "Envoyer" / "Annuler"
- Possibilité de modifier le montant avant envoi final

US-011 : Générer un avis de paiement pour un propriétaire

En tant qu'administrateur,
Je veux générer un avis de paiement,
Afin d'informer le propriétaire du versement à venir.

Différences avec quittance locataire :

- Titre : "AVIS DE PAIEMENT" au lieu de "QUITTANCE DE LOYER"
- Émetteur : Cosynight en tant que locataire
- Destinataire : Propriétaire
- Pas de RIB affiché (c'est le propriétaire qui reçoit l'argent)
- Date limite : 10 du mois (vs 5 pour locataires)

Critères d'acceptation :

- Template distinct de la quittance
- Même workflow de génération/envoi
- Réservé au rôle ADMIN

2.1.5 Archivage et historique

US-012 : Voir l'historique d'un locataire

En tant qu'utilisateur,
Je veux voir toutes les quittances envoyées à un locataire,
Afin de suivre les paiements.

Affichage :

- Tableau chronologique inversé (plus récent en haut)
- Colonnes : Date d'envoi, Période, Montant, Statut, Actions
- Action : Télécharger PDF / Renvoyer

Critères d'acceptation :

- Historique complet depuis la création du locataire
- Téléchargement du PDF original
- Possibilité de renvoyer un email (re-send)

US-013 : Tableau de bord des envois

En tant qu'utilisateur,
Je veux voir un récapitulatif des derniers envois,
Afin de suivre l'activité.

Affichage :

- Carte : Total quittances envoyées ce mois
- Carte : Total montants générés ce mois
- Liste : 10 dernières quittances envoyées
- Filtre par statut : Envoyées / Échec

Critères d'acceptation :

- Dashboard chargé en < 3 secondes
- KPIs mis à jour en temps réel

2.1.6 Paramètres de l'entreprise

US-014 : Modifier les informations de l'entreprise

En tant qu'administrateur,
Je veux modifier les informations de Cosynight,
Afin qu'elles apparaissent sur tous les documents générés.

Champs modifiables :

- Nom de l'entreprise
- Adresse complète
- SIRET
- Email de contact
- Téléphone

- Logo (upload image, max 2MB, formats : PNG, JPG, SVG)

Informations bancaires :

- IBAN complet
- Code banque
- Code guichet
- Numéro de compte
- Clé RIB
- Nom de la banque

Critères d'acceptation :

- 1 seule fiche entreprise (singleton)
- Modifications appliquées immédiatement sur tous nouveaux documents
- Logo redimensionné automatiquement pour le PDF
- Validation du format IBAN

US-015 : Gérer les templates d'email

En tant qu'administrateur,
Je veux personnaliser les emails envoyés,
Afin d'adapter le message aux destinataires.

Templates :

1. Email pour quittances locataires
2. Email pour avis de paiement propriétaires

Variables disponibles :

- {{FIRSTNAME}} : Prénom
- {{LASTNAME}} : Nom
- {{AMOUNT}} : Montant
- {{PERIOD_START}} : Date début période
- {{PERIOD_END}} : Date fin période
- {{DUE_DATE}} : Date limite de paiement
- {{MONTH}} : Nom du mois
- {{YEAR}} : Année

Critères d'acceptation :

- Éditeur WYSIWYG (TinyMCE ou équivalent)
- Prévisualisation avec données de test
- Variables auto-complétées lors de l'envoi

2.2 Fonctionnalités futures (Post-MVP)

Phase 2 (Mois 3-4)

US-100 : Envoi automatique programmé

Automatiser l'envoi des quittances chaque mois à une date définie (ex: le 1er)

US-101 : Gestion multi-utilisateurs

Créer/modifier/supprimer des comptes utilisateurs avec assignation de rôles

US-102 : Notifications

Alertes email/SMS pour échéances, échecs d'envoi, etc.

US-103 : Export comptable

Export Excel/CSV des quittances pour intégration comptable

Phase 3 (Mois 5-6)

US-200 : Génération de factures

Template de facturation pour prestations annexes

US-201 : Suivi des paiements

Marquer les quittances comme payées/impayées

US-202 : Relances automatiques

Email de relance automatique pour impayés

US-203 : Signature électronique

Signature numérique des documents (certificat SSL)

3. Spécifications techniques

3.1 Stack technologique retenue

Frontend

Framework: Next.js 14 (App Router)

Language: TypeScript 5.3+

Styling: TailwindCSS 3.4+

Components: shadcn/ui

Forms: react-hook-form + zod

Date: date-fns

Icons: lucide-react

Backend

Runtime: Node.js 20+ LTS

Framework: Next.js API Routes

ORM: Prisma 5.x

Type Safety: TypeScript end-to-end

Base de données

SGBD: MySQL 8.0+

Connexion: Prisma Client

Migrations: Prisma Migrate

Services externes

PDF Generation: @react-pdf/renderer

Email: nodemailer (Gmail SMTP temporaire)

Storage: MySQL (Base64) → AWS S3 (futur)

Authentication: NextAuth.js v5

Déploiement

Hébergement: Scaleway (à provisionner)

CI/CD: GitHub Actions

Monitoring: Sentry (optionnel)

Logs: Pino + File rotation

3.2 Exigences environnement

Développement

Node.js: >= 20.0.0

npm: >= 10.0.0

MySQL: >= 8.0.0

Git: >= 2.40.0

OS: macOS, Linux, Windows (WSL2)

Production

Serveur: 2 vCPU, 4GB RAM minimum

Stockage: 50GB SSD (évolutif)

Bande passante: 1TB/mois

SSL: Certificat Let's Encrypt

Backup: Quotidien automatique BDD

3.3 Configuration requise

Variables d'environnement

Database

DATABASE_URL="mysql://user:pass@host:3306/dbname"

NextAuth

NEXTAUTH_URL="https://rental.cosynight.fr"

NEXTAUTH_SECRET="[generated-secret-32-chars]"

Email (Gmail App Password)

SMTP_HOST="smtp.gmail.com"

SMTP_PORT=587

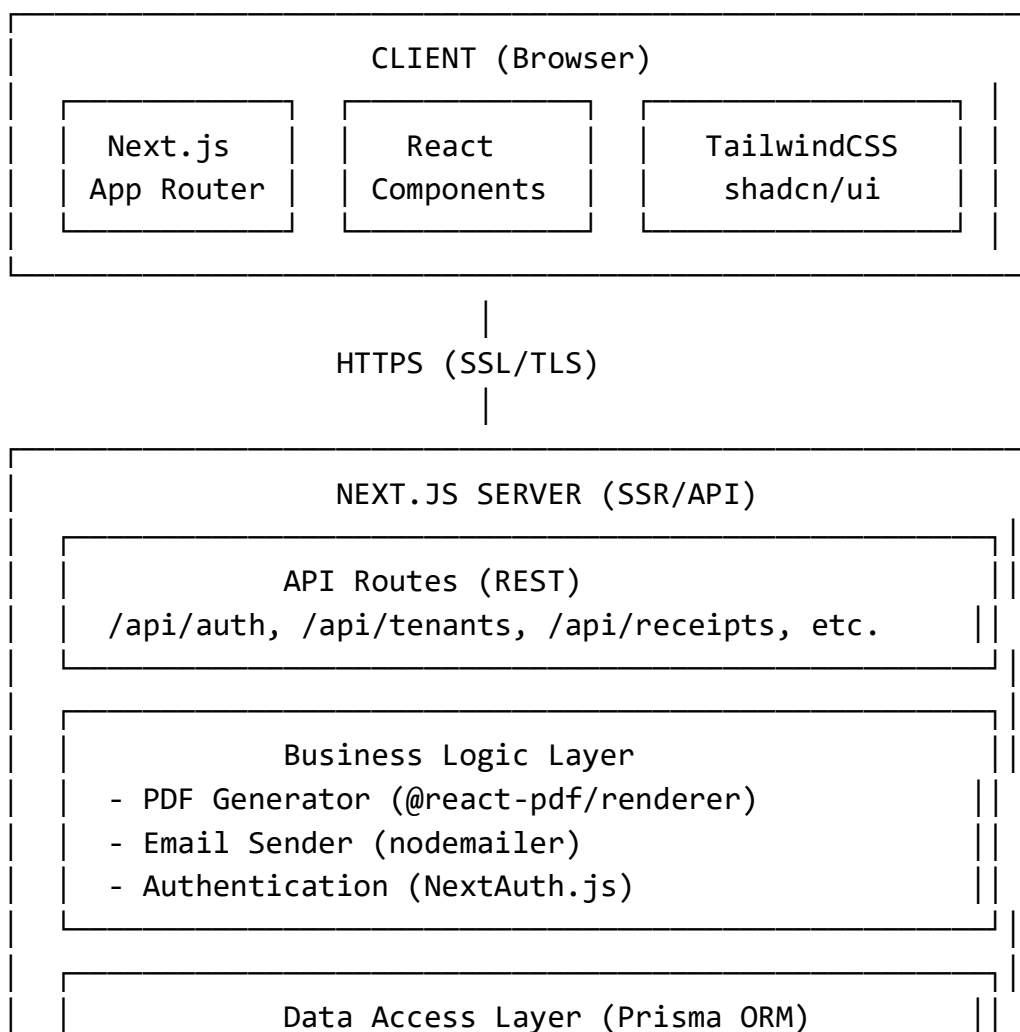
```
SMTP_USER="contact@cosynight.fr"
SMTP_PASSWORD="[app-password]"
SMTP_FROM="Cosynight <contact@cosynight.fr>"

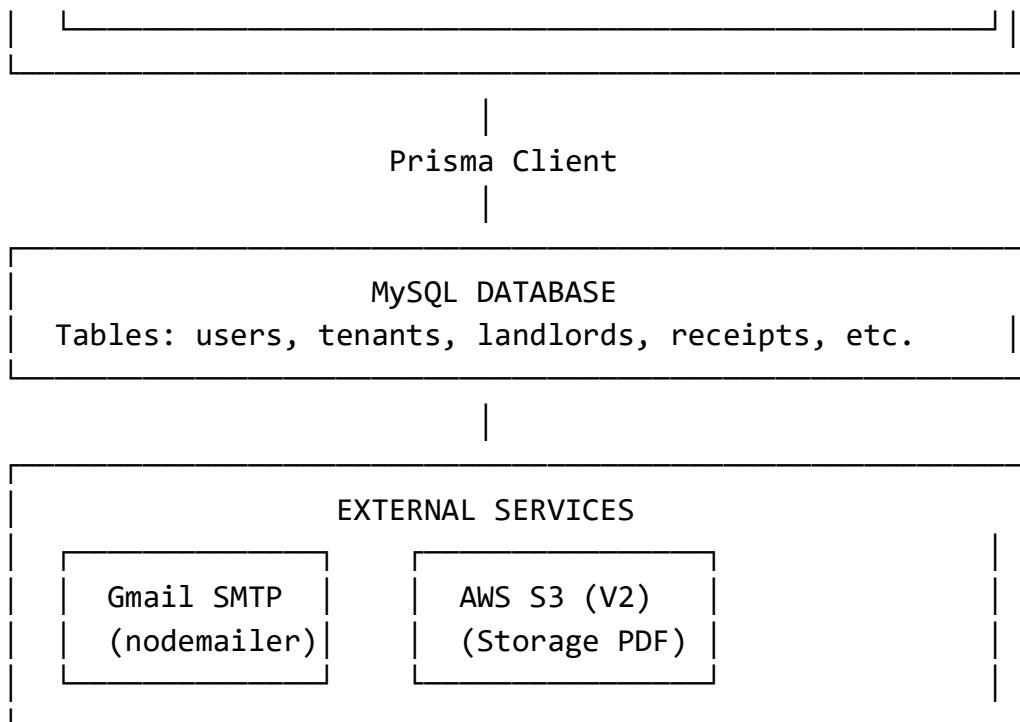
# App
NODE_ENV="production"
NEXT_PUBLIC_APP_URL="https://rental.cosynight.fr"

# Sentry (optionnel)
SENTRY_DSN="[url]"
```

4. Architecture système

4.1 Architecture applicative





4.2 Architecture de données

Relations entre entités

User (1) ————— (N) Sessions

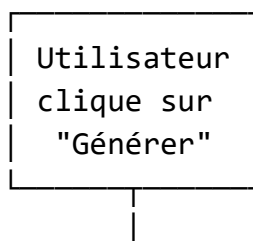
Tenant (1) ————— (N) Receipts

Landlord (1) ————— (N) Receipts

Company (1 singleton) — Utilisé par tous les Receipts

EmailTemplate (1 par type) — Utilisé lors de l'envoi

4.3 Flux de génération de quittance



v

1. Récupération des données
 - Locataire/Propriétaire
 - Company settings
 - Template email

v

2. Génération PDF
 - @react-pdf/renderer
 - Template quittance/avis
 - Render to Buffer

v

3. Stockage BDD
 - Base64 encode du PDF
 - INSERT dans table receipts

v

4. Envoi Email
 - Remplacement variables
 - Attachment PDF
 - nodemailer.sendMail()

v

5. Mise à jour statut
 - Status: SENT ou FAILED
 - sentAt: timestamp

v

6. Retour utilisateur
 - Toast notification

- Redirect vers historique

5. Modèle de données

5.1 Schéma Prisma complet

// Voir artifact "mvp_prisma_schema" pour version MVP
// Version complète avec toutes les fonctionnalités

```
model User {
  id          String    @id @default(cuid())
  email       String    @unique
  password    String    // bcrypt hash
  name        String
  role        Role      @default(USER)
  isActive    Boolean   @default(true)
  lastLoginAt DateTime?
  createdAt   DateTime  @default(now())
  updatedAt   DateTime  @updatedAt

  sessions    Session[]
  createdReceipts Receipt[] @relation("CreatedBy")

  @@index([email])
  @@map("users")
}

enum Role {
  ADMIN      // Accès complet
  COMPTABLE  // Lecture seule
  USER       // Gestion locataires uniquement
}

model Session {
  id          String    @id @default(cuid())
  userId      String
  token       String    @unique
  expiresAt   DateTime
```



```

    createdAt DateTime @default(now())

    user      User      @relation(fields: [userId], references: [id],
onDelete: Cascade)

    @@index([userId])
    @@index([token])
    @@map("sessions")
}

model Company {
  id          String    @id @default(cuid())
  name        String
  address     String    @db.Text
  siret       String
  email       String
  phone       String

  // Informations bancaires
  iban        String
  bankName    String
  bankCode    String    // Ex: 30087
  branchCode  String    // Ex: 33810
  accountNumber String  // Ex: 00022356802
  ribKey      String    // Ex: 55

  // Logo
  logoBase64  String?   @db.LongText
  logoFileName String?
  logoMimeType String?

  updatedAt   DateTime @updatedAt
  updatedBy   String?

  @@map("company")
}

model Tenant {
  id          String    @id @default(cuid())

  // Identité
  firstName   String

```

```

lastName      String
company       String?    // Si locataire professionnel

// Contact
email         String     @unique
phone        String
address       String     @db.Text

// Loyer
rentAmount    Decimal    @db.Decimal(10, 2)
entryDate     DateTime
exitDate      DateTime?  // Date de sortie si applicable

// Redevance (crédit pour le locataire)
creditAmount  Decimal?   @db.Decimal(10, 2)
creditReason  String?    @db.Text

// Métadonnées
isActive      Boolean    @default(true)
notes         String?    @db.Text
createdAt     DateTime    @default(now())
updatedAt     DateTime    @updatedAt
deletedAt     DateTime?   // Soft delete

// Relations
receipts      Receipt[]

@@index([email])
@@index([lastName])
@@index([isActive])
@@map("tenants")
}

model Landlord {
  id           String      @id @default(cuid())

  // Identité
  firstName    String
  lastName     String
  company      String?
  siret        String?

```

```

// Contact
email      String    @unique
phone      String
address    String    @db.Text

// Loyer que Cosynight paie
rentAmount Decimal   @db.Decimal(10, 2)

// Métadonnées
isActive   Boolean   @default(true)
notes      String?   @db.Text
createdAt  DateTime   @default(now())
updatedAt  DateTime   @updatedAt
deletedAt  DateTime?

// Relations
receipts    Receipt[]

@@index([email])
@@index([lastName])
@@index([isActive])
@@map("landlords")
}

enum ReceiptType {
    QUITTANCE      // Pour locataires
    AVIS_PAIEMENT  // Pour propriétaires
}

enum ReceiptStatus {
    DRAFT          // Brouillon
    SENT           // Envoyé avec succès
    FAILED         // Échec d'envoi
}

model Receipt {
    id          String    @id @default(cuid())
    type        ReceiptType
    status       ReceiptStatus @default(DRAFT)

    // Relation polymorphique (soit tenant, soit landlord)
    tenantId    String?

```

```

    tenant      Tenant?      @relation(fields: [tenantId],
references: [id], onDelete: Cascade)

    landlordId   String?
    landlord     Landlord?    @relation(fields: [landlordId],
references: [id], onDelete: Cascade)

    // Données financières
    amount       Decimal      @db.Decimal(10, 2)
    periodStart   DateTime     // 1er du mois
    periodEnd     DateTime     // Dernier jour du mois
    dueDate       DateTime     // 5 ou 10 du mois selon type

    // PDF (stockage temporaire en Base64)
    pdfBase64     String       @db.LongText
    pdfFileName   String       // Ex:
quittance_2025_11_kouame_tiffany.pdf

    // S3 URL (futur)
    pdfS3Url      String?

    // Email
    emailSubject  String?
    emailBody     String?      @db.Text
    sentAt        DateTime?
    sentTo        String?      // Email du destinataire

    // Métadonnées
    createdById   String?
    createdBy     User?        @relation("CreatedBy", fields:
[createdById], references: [id], onDelete: SetNull)
    createdAt     DateTime     @default(now())

    @@index([tenantId])
    @@index([landlordId])
    @@index([periodStart])
    @@index([status])
    @@index([createdAt])
    @@map("receipts")
}

model EmailTemplate {

```

```

    id          String      @id @default(cuid())
    type        ReceiptType @unique
    subject     String
    body        String      @db.Text

    updatedAt   DateTime    @updatedAt

    @@map("email_templates")
}

model AuditLog {
  id          String      @id @default(cuid())
  userId      String?
  action      String      // CREATE, UPDATE, DELETE, SEND_EMAIL
  entityType  String      // Tenant, Landlord, Receipt, etc.
  entityId    String
  changes     Json?       // Snapshot avant/après
  ipAddress   String?
  userAgent   String?
  createdAt   DateTime    @default(now())

  @@index([userId])
  @@index([entityType, entityId])
  @@index([createdAt])
  @@map("audit_logs")
}

```

5.2 Contraintes d'intégrité

-- Contraintes CHECK (à ajouter via migration raw SQL)

```

ALTER TABLE tenants
ADD CONSTRAINT check_rent_positive
CHECK (rentAmount > 0);

ALTER TABLE landlords
ADD CONSTRAINT check_rent_positive
CHECK (rentAmount > 0);

ALTER TABLE receipts
ADD CONSTRAINT check_amount_positive

```

```

CHECK (amount > 0);

ALTER TABLE receipts
ADD CONSTRAINT check_period_valid
CHECK (periodEnd > periodStart);

-- Contrainte polymorphe (soit tenant, soit landlord, pas les
deux)
ALTER TABLE receipts
ADD CONSTRAINT check_recipient_xor
CHECK (
    (tenantId IS NOT NULL AND landlordId IS NULL) OR
    (tenantId IS NULL AND landlordId IS NOT NULL)
);

```

5.3 Index de performance

```

-- Index pour optimiser les requêtes fréquentes

-- Recherche de locataires/propriétaires actifs
CREATE INDEX idx_tenants_active_lastname ON tenants(isActive,
lastName);
CREATE INDEX idx_landlords_active_lastname ON landlords(isActive,
lastName);

-- Historique des quittances par période
CREATE INDEX idx_receipts_period ON receipts(periodStart,
periodEnd);

-- Dashboard : dernières quittances envoyées
CREATE INDEX idx_receipts_sent ON receipts(status, sentAt DESC);

-- Statistiques mensuelles
CREATE INDEX idx_receipts_month ON receipts(periodStart, status);

```

6. Interfaces utilisateur

6.1 Charte graphique

Couleurs principales

```
/* Palette Cosynight */
--primary: #2563eb;      /* Bleu principal */
--primary-dark: #1e40af; /* Bleu foncé */
--secondary: #64748b;    /* Gris-bleu */
--success: #10b981;      /* Vert succès */
--warning: #f59e0b;      /* Orange alerte */
--error: #ef4444;        /* Rouge erreur */
--background: #f8fafc;   /* Gris clair fond */
--surface: #ffffff;      /* Blanc cartes */
--text: #0f172a;         /* Noir texte */
--text-muted: #64748b;   /* Gris texte secondaire */
```

Typographie

```
/* Fonte principale */
font-family: 'Inter', -apple-system, BlinkMacSystemFont, sans-serif;
```

```
/* Tailles */
--text-xs: 0.75rem;  /* 12px */
--text-sm: 0.875rem; /* 14px */
--text-base: 1rem;   /* 16px */
--text-lg: 1.125rem; /* 18px */
--text-xl: 1.25rem;  /* 20px */
--text-2xl: 1.5rem;  /* 24px */
--text-3xl: 1.875rem; /* 30px */
```

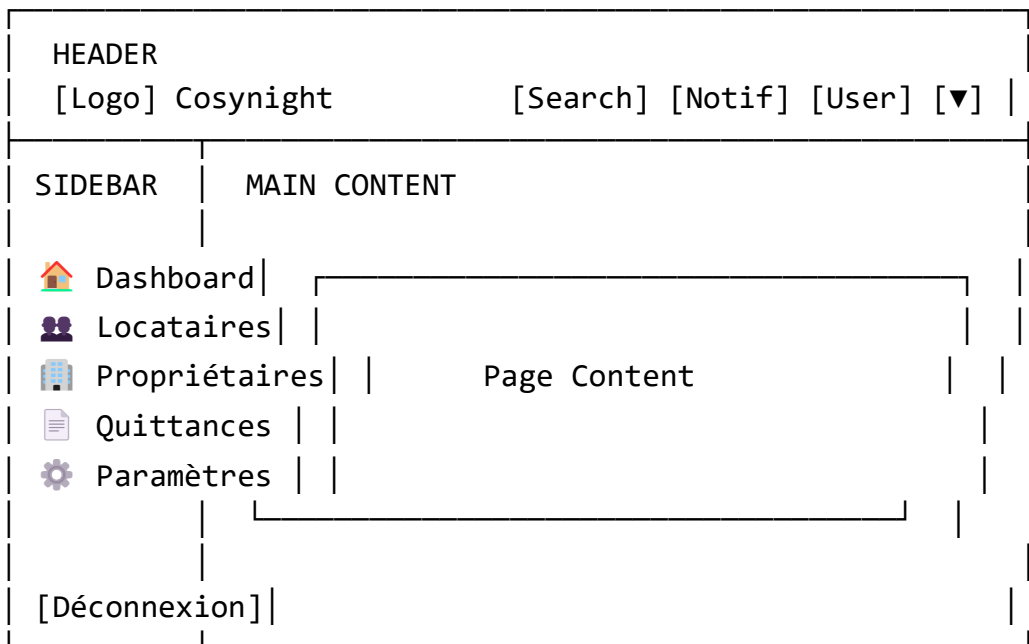
Espacements

```
/* Marges et padding */
--spacing-1: 0.25rem; /* 4px */
--spacing-2: 0.5rem;  /* 8px */
--spacing-3: 0.75rem; /* 12px */
--spacing-4: 1rem;    /* 16px */
--spacing-6: 1.5rem;  /* 24px */
```

--spacing-8: 2rem; /* 32px */

6.2 Structure des pages

Layout principal (Dashboard)



Page Login

[Logo Cosynight]	
Plateforme de Quittances	
Connexion	
Email:	
Mot de passe:	
☐	Se souvenir de moi
Se connecter	

Mot de passe oublié ?

Page Dashboard

Tableau de bord

12
Quittances
ce mois

18 870€
Total
ce mois

2
En attente

Dernières quittances envoyées

Date	Destinataire	Montant	Actions
28/10/2025	KOUAME Tiffany	1570€	[📄][✉️]
28/10/2025	DUPONT Marc	1200€	[📄][✉️]
27/10/2025	MARTIN Sophie	950€	[📄][✉️]

Page Liste Locataires

Locataires

[+ Nouveau locataire]

[🔍 Rechercher...] [Filtrer ▼] [Trier ▼]

Nom	Email	Loyer	Actions
KOUAME T.	tiffany@...	1570€	[👁️][✏️][📄]
DUPONT M.	marc@...	1200€	[👁️][✏️][📄]
MARTIN S.	sophie@...	950€	[👁️][✏️][📄]

Pagination: [◀] 1 2 3 [▶]

Modal Créer Locataire

Nouveau locataire		[X]
Prénom *	[_____]	
Nom *	[_____]	
Email *	[_____]	
Téléphone *	[_____]	
Adresse complète *		
[_____]		
[_____]		
Montant du loyer *	[_____] €	
Date d'entrée *	[📅 01/04/2025]	
— Optionnel —		
Société	[_____]	
Redevance	[_____] €	
Motif redevance	[_____]	
[Annuler] [Créer le locataire]		

Modal Générer Quittance

Générer une quittance	[X]
Locataire: KOUAME Tiffany	
tiffany.kouame@example.com	

Période *	
Mois: [Novembre ▼]	Année: [2025 ▼]
Du [01/11/2025] au [30/11/2025]	
Montant du loyer:	1570,00 €
Redevance:	-0,00 €
Total:	1570,00 €
[Annuler] [Prévisualiser]	

Modal Prévisualisation PDF

Prévisualisation	[X]
[Visualisation du PDF] QUITTANCE DE LOYER COSYNIGHT ...	
[Télécharger PDF] [Annuler] [Envoyer par email]	

6.3 Composants réutilisables (shadcn/ui)

- // Liste des composants à installer
- Button (variant: default, destructive, outline, ghost)
 - Input (text, email, tel, number, date)
 - Form (react-hook-form integration)
 - Table (sortable, avec pagination)

- Dialog (Modal)
- Select (Dropdown)
- Badge (Status indicators)
- Card (Containers)
- Toast (Notifications)
- Avatar (User profile)
- Dropdown Menu (Actions menu)
- Tabs
- Separator
- Label
- Textarea

7. Parcours utilisateurs

7.1 Parcours : Créer et envoyer une quittance

Acteur : Utilisateur (USER ou ADMIN)

Prérequis :

- Utilisateur connecté
- Au moins 1 locataire créé
- Informations entreprise renseignées

Étapes :

1. Accès à la fonctionnalité

- a. Clic sur "Locataires" dans le menu
- b. Recherche du locataire concerné (optionnel)
- c. Clic sur bouton "Générer quittance" (icône )

2. Saisie des informations

- a. Modal "Générer une quittance" s'ouvre
- b. Nom du locataire pré-affiché (lecture seule)
- c. Sélection du mois (dropdown)
- d. Sélection de l'année (dropdown)
- e. Dates de période calculées automatiquement
- f. Montant pré-rempli avec le loyer du locataire
- g. Possibilité de modifier le montant si besoin
- h. Clic sur "Prévisualiser"

3. Prévisualisation

- a. Génération du PDF (2-3 secondes)
- b. Affichage dans un viewer intégré
- c. Vérification visuelle du document
- d. Options : Télécharger / Annuler / Envoyer

4. Envoi

- a. Clic sur "Envoyer par email"
- b. Loading spinner pendant l'envoi
- c. Email envoyé avec PDF en pièce jointe
- d. Sauvegarde dans la base de données

5. Confirmation

- a. Toast de succès : "Quittance envoyée avec succès à tiffany@example.com"
- b. Modal se ferme automatiquement
- c. Badge "Quittances: 5" mis à jour dans la liste

Cas d'erreur :

- Email invalide → Message "Adresse email invalide"
- Erreur SMTP → "Échec d'envoi. Vérifier les paramètres SMTP"
- Quittance déjà générée pour ce mois → Avertissement avec option de régénérer

Durée estimée : 30 secondes

7.2 Parcours : Consulter l'historique d'un locataire

Acteur : Utilisateur (tous rôles)

Étapes :

1. Accès à la liste des locataires
2. Clic sur l'icône "👁 Voir détails"
3. Page détails du locataire s'affiche avec :
 - a. Informations personnelles en haut
 - b. Onglet "Historique des quittances"
4. Tableau des quittances envoyées :
 - a. Date d'envoi
 - b. Période
 - c. Montant
 - d. Statut (badge vert "Envoyée")

- e. Actions : Télécharger PDF / Renvoyer email
- 5. Clic sur "Télécharger" → PDF s'ouvre dans un nouvel onglet

Durée estimée : 15 secondes

7.3 Parcours : Modifier les informations de l'entreprise

Acteur : Administrateur (ADMIN)

Prérequis : Rôle ADMIN

Étapes :

1. Clic sur "  Paramètres" dans le menu
2. Onglet "Informations entreprise" (par défaut)
3. Formulaire pré-rempli avec les données actuelles :
 - a. Nom, adresse, SIRET, email, téléphone
 - b. IBAN, codes bancaires
 - c. Logo actuel affiché
4. Modification des champs souhaités
5. Upload d'un nouveau logo (optionnel) :
 - a. Drag & drop ou clic
 - b. Formats acceptés : PNG, JPG, SVG
 - c. Taille max : 2 MB
 - d. Preview immédiat
6. Clic sur "Enregistrer les modifications"
7. Toast de succès : "Informations mises à jour"
8. Tous les prochains documents générés utiliseront ces nouvelles infos

Durée estimée : 2 minutes

7.4 Parcours : Premier login et découverte

Acteur : Nouvel utilisateur

Étapes :

1. **Login**
 - a. Accès à l'URL : <https://rental.cosynight.fr>
 - b. Saisie email et mot de passe fournis

- c. Clic sur "Se connecter"

2. Dashboard vide

- a. Message d'accueil : "Bienvenue ! Commencez par créer votre premier locataire"
- b. Bouton CTA : "+ Créer un locataire"

3. Création du premier locataire

- a. Clic sur le bouton CTA
- b. Remplissage du formulaire
- c. Sauvegarde

4. Génération de la première quittance

- a. Message : "Votre locataire a été créé. Générer sa première quittance ?"
- b. Bouton : "Générer maintenant"
- c. Suivi du parcours standard

5. Découverte des fonctionnalités

- a. Tooltip sur les icônes importantes
- b. Lien vers documentation (optionnel)

8. Exigences de sécurité

8.1 Authentification

Mot de passe

// Règles de sécurité

- Longueur minimale : 8 caractères
- Obligatoire : 1 majuscule, 1 minuscule, 1 chiffre
- Hashage : bcrypt avec salt de 10 rounds
- Stockage : Jamais en clair, uniquement hash en BDD

Session

// Configuration NextAuth

- Strategy : JWT
- Durée de vie : 7 jours (avec remember me)
- Refresh automatique : avant expiration
- Révocation : Déconnexion supprime le token
- CSRF Protection : Activée par défaut

Récupération de mot de passe

Phase 2 :

- Email avec lien de réinitialisation
- Token unique valide 1 heure
- Lien à usage unique (invalidé après utilisation)

8.2 Autorisation (RBAC)

Matrice des permissions

Fonctionnalité	ADMIN	COMPTABLE	USER
Voir dashboard	✓	✓	✓
Créer locataire	✓	✗	✓
Modifier locataire	✓	✗	✓
Supprimer locataire	✓	✗	✗
Voir locataires	✓	✓	✓
Créer propriétaire	✓	✗	✗
Voir propriétaires	✓	✓	✗
Générer quittance locataire	✓	✗	✓
Générer avis paiement	✓	✗	✗
Télécharger PDF	✓	✓	✓
Modifier paramètres entreprise	✓	✗	✗
Modifier templates email	✓	✗	✗
Créer utilisateurs	✓	✗	✗
Voir audit logs	✓	✗	✗

Implémentation

```
// Middleware Next.js
export function middleware(req: NextRequest) {
  const token = getToken(req);
  const path = req.nextUrl.pathname;

  // Routes réservées ADMIN
  const adminRoutes = ['/landlords', '/settings', '/users'];
  if (adminRoutes.some(route => path.startsWith(route))) {
    if (token?.role !== 'ADMIN') {
```



```
        return NextResponse.redirect('/dashboard');
    }
}

return NextResponse.next();
}
```

8.3 Protection des données

RGPD

****Données personnelles collectées :****

- Identité : Nom, prénom
- Contact : Email, téléphone, adresse
- Financières : Montant du loyer, historique des paiements

****Droits des utilisateurs :****

- Droit d'accès : Export de toutes les données sur demande
- Droit de rectification : Modification via l'interface
- Droit à l'effacement : Soft delete avec anonymisation après 10 ans
- Droit à la portabilité : Export JSON/CSV

****Conservation :****

- Documents comptables : 10 ans (obligation légale)
- Données personnelles après fin de bail : 10 ans puis anonymisation
- Logs d'audit : 1 an

****Mesures techniques :****

- Chiffrement en transit : HTTPS/TLS 1.3
- Chiffrement au repos : Encryption base de données
- Accès restreints : Authentification + RBAC
- Traçabilité : Logs d'audit de toutes les actions

Validation des inputs

```
// Avec Zod
import { z } from 'zod';

const tenantSchema = z.object({
  firstName: z.string()
```

```

    .min(1, "Prénom requis")
    .max(50)
    .regex(/^[a-zA-ZÀ-ÿ\s-]+$/, "Caractères invalides"),

email: z.string()
    .email("Email invalide")
    .toLowerCase(),

phone: z.string()
    .regex(/^(?:(?:\+|00)33|0)\s*[1-9](?:[\s.-]*\d{2}){4}$/,
        "Format invalide (ex: 06 12 34 56 78)"),

rentAmount: z.number()
    .positive("Doit être > 0")
    .max(100000, "Montant trop élevé"),
});

```

Protection contre les attaques

```

// XSS : Sanitization automatique par React
// SQL Injection : Prisma ORM (requêtes préparées)
// CSRF : NextAuth.js (tokens CSRF automatiques)
// Rate Limiting : À implémenter

import rateLimit from 'express-rate-limit';

const limiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutes
  max: 100, // 100 requêtes max
  message: 'Trop de requêtes, réessayez plus tard'
});

// Appliquer sur routes sensibles (/api/receipts/generate)

```

9. Performances et scalabilité

9.1 Objectifs de performance

Métrique	Cible	Critique
Time to First Byte (TTFB)	< 500ms	< 1s
First Contentful Paint (FCP)	< 1.5s	< 3s
Largest Contentful Paint (LCP)	< 2.5s	< 4s
Génération PDF	< 3s	< 5s
Envoi email	< 5s	< 10s
Chargement liste (50 items)	< 1s	< 2s
Recherche (debounce)	< 500ms	< 1s

9.2 Optimisations

Frontend

```
// Code splitting automatique (Next.js App Router)
// Lazy loading des composants lourds
const PDFViewer = dynamic(() => import('@/components/PDFViewer'), {
  loading: () => <Skeleton />,
  ssr: false
});

// Images optimisées
import Image from 'next/image';
<Image
  src="/logo.png"
  width={100}
  height={100}
  alt="Logo"
  priority // Pour le logo en header
/>

// Pagination côté serveur
const ITEMS_PER_PAGE = 50;
```

Backend

```
// Prisma : Select uniquement les champs nécessaires
const tenants = await prisma.tenant.findMany({
  select: {
    id: true,
    firstName: true,
    lastName: true,
    email: true,
    rentAmount: true,
    _count: {
      select: { receipts: true }
    }
  },
  where: { isActive: true },
  take: ITEMS_PER_PAGE,
  skip: page * ITEMS_PER_PAGE
});

// Cache des paramètres entreprise (rarement modifiés)
import { cache } from 'react';

export const getCompanySettings = cache(async () => {
  return await prisma.company.findUnique({
    where: { id: 'default' }
  });
});
```

Base de données

```
-- Index sur colonnes fréquemment filtrées
CREATE INDEX idx_tenants_search ON tenants(lastName, firstName,
email);
CREATE INDEX idx_receipts_recent ON receipts(createdAt DESC,
status);

-- Analyse des requêtes lentes
EXPLAIN SELECT * FROM receipts WHERE tenantId = 'xxx';
```

9.3 Scalabilité

Capacité actuelle (MVP)

- Utilisateurs concurrents : 10-20
- Locataires/Propriétaires : 500 max
- Quittances/mois : 500-1000
- Stockage : 10 GB (PDFs en base64)
- Bande passante : 100 GB/mois

Évolution prévue (Phase 2-3)

- Migration PDFs vers S3 : Stockage illimité
- CDN pour assets statiques : Temps de chargement réduit
- Job queue (Bull/Redis) : Envois batch asynchrones
- Read replicas MySQL : Lecture optimisée
- Horizontal scaling : Plusieurs instances Next.js derrière load balancer

10. Contraintes et dépendances

10.1 Contraintes techniques

Obligatoires

- **Next.js 14+** : App Router obligatoire (Server Components)
- **TypeScript** : Typage strict activé
- **MySQL 8.0+** : Pas de SQLite ou PostgreSQL
- **Email SMTP** : Gmail temporaire, puis service professionnel (Resend/SendGrid)
- **HTTPS** : Obligatoire en production (Let's Encrypt)

Recommandées

- Node.js 20 LTS (support jusqu'en 2026)
- Prisma 5.x (dernière version stable)
- TailwindCSS 3.4+ (meilleure DX)

- React 18+ (Concurrent features)

10.2 Dépendances externes

Services tiers

Service	Usage	Criticité	Fallback
Gmail SMTP	Envoi emails MVP	Élevée	File d'attente locale
Scaleway	Hébergement	Critique	-
Let's Encrypt	Certificat SSL	Critique	-
npm registry	Dépendances	Moyenne	Cache local

Librairies critiques

```
{
  "dependencies": {
    "next": "^14.0.0",
    "react": "^18.2.0",
    "prisma": "^5.7.0",
    "@prisma/client": "^5.7.0",
    "next-auth": "^5.0.0-beta",
    "@react-pdf/renderer": "^3.1.0",
    "nodemailer": "^6.9.0",
    "zod": "^3.22.0",
    "react-hook-form": "^7.49.0",
    "date-fns": "^3.0.0"
  }
}
```

10.3 Contraintes métier

Légales

- Conservation documents comptables : 10 ans minimum
- Mentions légales obligatoires sur les quittances
- RGPD : Consentement pour traitement des données
- Numéro SIRET valide sur les documents

Fonctionnelles

- 1 seule quittance par locataire par mois (unicité)
- Montant du loyer ne peut être négatif
- Date limite de paiement : 5 du mois (locataires), 10 du mois (propriétaires)
- Email unique par locataire/propriétaire
- Logo entreprise : max 2MB, formats PNG/JPG/SVG

11. Planning de réalisation

11.1 Macro-planning (8 semaines)

Semaine 1-2 : Setup + Auth + Locataires

- └ Jour 1-2 : Setup projet + BDD + Seed
- └ Jour 3-4 : Authentification NextAuth
- └ Jour 5-6 : API Locataires (CRUD)
- └ Jour 7-8 : UI Liste/Formulaire Locataires
- └ Jour 9-10 : Tests + Corrections

Semaine 3-4 : Génération PDF + Envoi Email

- └ Jour 11-12 : Template PDF (@react-pdf)
- └ Jour 13-14 : Service Email (nodemailer)
- └ Jour 15-16 : API Génération + Envoi
- └ Jour 17-18 : UI Modal génération + Preview
- └ Jour 19-20 : Tests end-to-end

Semaine 5 : Propriétaires + Avis Paiement

- └ Jour 21-22 : API Propriétaires (copie Locataires)
- └ Jour 23-24 : Template Avis de Paiement
- └ Jour 25 : UI Propriétaires + Restrictions ADMIN

Semaine 6 : Historique + Paramètres

- └ Jour 26-27 : API/UI Historique des quittances
- └ Jour 28-29 : API/UI Paramètres entreprise
- └ Jour 30 : Upload logo + Preview

Semaine 7 : Dashboard + Polish

- └ Jour 31-32 : Dashboard avec KPIs

- |— Jour 33-34 : Recherche + Filtres + Tri
- |— Jour 35 : Responsive mobile

Semaine 8 : Tests + Déploiement

- |— Jour 36-37 : Tests utilisateurs + Corrections bugs
- |— Jour 38-39 : Documentation + Guide utilisateur
- |— Jour 40-41 : Setup Scaleway + CI/CD
- |— Jour 42 : Déploiement production
- |— Jour 43-44 : Formation client + Recette

11.2 Planning détaillé Sprint 1 (Exemple)

Sprint 1 : Setup + Auth + Locataires (10 jours)

Jour 1-2 : Initialisation projet

Tâches :

- ☐ Créer repository GitHub
- ☐ Initialiser Next.js 14 + TypeScript
- ☐ Installer dépendances (Prisma, NextAuth, shadcn/ui)
- ☐ Configurer TailwindCSS
- ☐ Setup base de données MySQL
- ☐ Créer schéma Prisma (tables User, Company, Tenant)
- ☐ Générer et appliquer migrations
- ☐ Créer seed.ts avec données de test
- ☐ Configurer .env.local
- ☐ Premier commit

Livrables :

- Repository GitHub initialisé
- Projet Next.js fonctionnel en local
- Base de données créée et seedée
- Documentation README.md

Jour 3-4 : Authentification

Tâches :

- ☐ Configurer NextAuth.js v5
- ☐ Créer route API /api/auth/[...nextauth]
- ☐ Créer page /login avec formulaire
- ☐ Implémenter CredentialsProvider
- ☐ Configurer sessions JWT
- ☐ Créer middleware pour routes protégées
- ☐ Tester login/logout
- ☐ Gérer les erreurs d'authentification

Livrables :

- Page de connexion fonctionnelle
- Session persistante
- Redirection automatique si non connecté

Jour 5-6 : API Locataires

Tâches :

- ☐ Créer route GET /api/tenants (liste)
- ☐ Créer route POST /api/tenants (création)
- ☐ Créer route GET /api/tenants/[id] (détails)
- ☐ Créer route PATCH /api/tenants/[id] (modification)
- ☐ Créer route DELETE /api/tenants/[id] (soft delete)
- ☐ Validation Zod pour tous les inputs
- ☐ Gestion des erreurs HTTP
- ☐ Tester avec Postman/Thunder Client

Livrables :

- API REST complète pour locataires
- Validation des données
- Documentation API (commentaires)

Jour 7-8 : Interface Locataires

Tâches :

- ☐ Installer composants shadcn : Table, Dialog, Form, Button
- ☐ Créer page /dashboard/tenants

- ☐ Créer composant TenantsList (tableau)
- ☐ Créer composant TenantForm (modal création)
- ☐ Implémenter formulaire avec react-hook-form
- ☐ Ajouter recherche en temps réel
- ☐ Ajouter pagination
- ☐ Gérer les états de chargement (Skeleton)
- ☐ Afficher toasts pour succès/erreurs

Livrables :

- Page liste locataires complète
- Formulaire de création fonctionnel
- UX fluide avec feedback visuel

Jour 9-10 : Tests et corrections

Tâches :

- ☐ Tests manuels de tous les parcours
- ☐ Corriger bugs identifiés
- ☐ Améliorer validation des formulaires
- ☐ Optimiser requêtes Prisma
- ☐ Refactoring du code
- ☐ Commenter le code complexe
- ☐ Mettre à jour le README

Livrables :

- Module locataires stable et testé
- Code propre et documenté
- Prêt pour démo au client

11.3 Jalons et démos

Jalon 1 : Fin Sprint 1 (Semaine 2)

Date : Jour 10

Démo :

- Connexion fonctionnelle
- Création et gestion de locataires
- Interface complète

Critères de validation :

- ☒ Admin peut se connecter
- ☒ Créer un locataire avec toutes les données
- ☒ Voir la liste des locataires
- ☒ Modifier un locataire
- ☒ Désactiver un locataire

Jalon 2 : Fin Sprint 2 (Semaine 4)

Date : Jour 20

Démo :

- Génération de quittance PDF
- Envoi automatique par email
- Archivage

Critères de validation :

- ☒ Générer un PDF conforme au template
- ☒ Envoyer le PDF par email
- ☒ PDF sauvegardé en base de données
- ☒ Historique visible dans le profil locataire

Jalon 3 : Fin Sprint 3 (Semaine 5)

Date : Jour 25

Démo :

- Gestion des propriétaires (ADMIN uniquement)
- Avis de paiement

Critères de validation :

- ☒ ADMIN peut créer des propriétaires
- ☒ USER ne voit pas l'onglet propriétaires
- ☒ Avis de paiement généré avec le bon template
- ☒ Différences appliquées (date limite, pas de RIB)

Jalon 4 : Fin Sprint 4 (Semaine 6)

Date : Jour 30

Démo :

- Paramètres entreprise modifiables
- Dashboard fonctionnel

Critères de validation :

- ☒ Modifier nom, adresse, SIRET
- ☒ Upload et affichage du logo
- ☒ Modifications visibles sur nouveaux documents
- ☒ Dashboard affiche statistiques correctes

Jalon 5 : MVP Complet (Semaine 7)

Date : Jour 35

Démo :

- Application complète et polie
- Responsive

Critères de validation :

- ☒ Toutes les fonctionnalités MVP fonctionnent
- ☒ UI responsive (mobile/tablet/desktop)
- ☒ Performance optimale
- ☒ Aucun bug bloquant

Jalon 6 : Production (Semaine 8)

Date : Jour 44

Livraison finale :

- Application déployée sur Scaleway
- Formation client effectuée
- Documentation fournie

Critères de validation :

-  Application accessible en ligne (HTTPS)
-  Client formé et autonome
-  Guide utilisateur fourni
-  Recette signée

11.4 Rétrospectives hebdomadaires

Chaque vendredi à 16h00 (30 minutes)

Format :

Rétrospective Semaine N

 Ce qui a bien fonctionné

- [Point positif 1]
- [Point positif 2]

 Difficultés rencontrées

- [Problème 1] → Solution apportée
- [Problème 2] → En cours

 Avancement

- Prévu : 10 tâches
- Réalisé : 8 tâches
- Delta : -2 tâches (à reporter)

 Objectifs semaine prochaine

- [Tâche prioritaire 1]
- [Tâche prioritaire 2]

🚨 Risques identifiés

- [Risque 1] : Probabilité, Impact, Mitigation

12. Critères de recette

12.1 Tests fonctionnels (Checklist)

Authentication

- ❑ TC-001 : Login avec identifiants valides → Succès
- ❑ TC-002 : Login avec email invalide → Erreur affichée
- ❑ TC-003 : Login avec mot de passe incorrect → Erreur affichée
- ❑ TC-004 : Session maintenue après refresh page
- ❑ TC-005 : Déconnexion efface la session
- ❑ TC-006 : Accès direct à /dashboard sans login → Redirect /login

Gestion Locataires

- ❑ TC-010 : Créer un locataire avec toutes les données → Succès
- ❑ TC-011 : Créer un locataire avec email déjà existant → Erreur
- ❑ TC-012 : Créer un locataire avec montant négatif → Erreur
- ❑ TC-013 : Modifier un locataire → Modifications sauvegardées
- ❑ TC-014 : Désactiver un locataire → N'apparaît plus dans liste active
- ❑ TC-015 : Rechercher un locataire par nom → Résultats filtrés
- ❑ TC-016 : Tri par nom/loyer/date → Ordre correct
- ❑ TC-017 : Pagination fonctionne si > 50 locataires

Génération Quittances

- ❑ TC-020 : Générer quittance avec données valides → PDF créé
- ❑ TC-021 : PDF contient toutes les informations correctes
- ❑ TC-022 : PDF utilise le logo de l'entreprise
- ❑ TC-023 : Email envoyé avec PDF en pièce jointe
- ❑ TC-024 : Quittance sauvegardée dans l'historique
- ❑ TC-025 : Impossible de générer 2 fois pour même mois →

Avertissement

- TC-026 : Prévisualisation PDF fonctionne
- TC-027 : Téléchargement PDF fonctionne
- TC-028 : Renvoyer une quittance existante → Nouvel email envoyé

Gestion Propriétaires (ADMIN)

- TC-030 : USER ne voit pas l'onglet propriétaires
- TC-031 : ADMIN peut créer un propriétaire
- TC-032 : ADMIN peut générer un avis de paiement
- TC-033 : Avis de paiement n'affiche pas de RIB
- TC-034 : Date limite sur avis = 10 du mois
- TC-035 : Titre du document = "AVIS DE PAIEMENT"

Paramètres Entreprise

- TC-040 : USER ne peut pas accéder aux paramètres
- TC-041 : ADMIN peut modifier nom entreprise
- TC-042 : ADMIN peut modifier IBAN
- TC-043 : Upload logo PNG → Logo affiché
- TC-044 : Upload logo > 2MB → Erreur
- TC-045 : Upload fichier .exe → Erreur
- TC-046 : Modifications appliquées sur prochaine quittance
- TC-047 : Validation format IBAN (27 caractères)

Dashboard

- TC-050 : KPI "Quittances ce mois" → Nombre correct
- TC-051 : KPI "Total ce mois" → Somme correcte
- TC-052 : Liste des 10 dernières quittances → Ordre chronologique
- TC-053 : Clic sur une quittance → Télécharge le PDF

12.2 Tests non-fonctionnels

Performance

- TNF-001 : Chargement page login < 2s
- TNF-002 : Chargement liste 50 locataires < 2s
- TNF-003 : Génération PDF < 5s
- TNF-004 : Envoi email < 10s
- TNF-005 : Recherche temps réel < 500ms
- TNF-006 : Upload logo 1MB < 3s

Sécurité

- TNF-010 : Mots de passe hashés en BDD (bcrypt)
- TNF-011 : Pas de données sensibles en clair dans logs
- TNF-012 : HTTPS activé en production
- TNF-013 : Headers sécurisés (HSTS, CSP)
- TNF-014 : Protection CSRF activée
- TNF-015 : Validation inputs côté serveur
- TNF-016 : Pas d'injection SQL possible (Prisma ORM)

Compatibilité

- TNF-020 : Chrome 100+ → OK
- TNF-021 : Firefox 100+ → OK
- TNF-022 : Safari 15+ → OK
- TNF-023 : Edge 100+ → OK
- TNF-024 : Mobile iOS Safari → OK
- TNF-025 : Mobile Android Chrome → OK
- TNF-026 : Responsive < 768px → Layout mobile
- TNF-027 : Responsive 768-1024px → Layout tablet

Accessibilité (WCAG 2.1 AA)

- TNF-030 : Contraste texte/fond $\geq 4.5:1$
- TNF-031 : Navigation au clavier possible
- TNF-032 : Focus visible sur éléments interactifs
- TNF-033 : Labels sur tous les champs de formulaire
- TNF-034 : Messages d'erreur explicites

- TNF-035 : Textes alternatifs sur images

12.3 Tests de régression

À exécuter avant chaque livraison :

1. Smoke Test (15 min) :
 - Login/Logout
 - Créer 1 locataire
 - Générer 1 quittance
 - Télécharger le PDF
 - Si échec : STOP, corriger avant continuer
2. Parcours critiques (1h) :
 - Tous les TC-0XX listés ci-dessus
 - Focus sur parcours métier principaux
3. Tests exploratoires (30 min) :
 - Tester des cas limites
 - Essayer de "casser" l'application
 - Vérifier cohérence globale UX

12.4 Recette finale (Client)

Grille d'évaluation pour validation finale :

Critère	Poids	Note /5	Total
Fonctionnalités			
Gestion locataires complète	20%		
Génération quittances conforme	25%		
Envoi emails automatique	15%		
Archivage et historique	10%		
Paramètres modifiables	10%		

Qualité

Interface intuitive	10%	
Performance	5%	
Fiabilité (pas de bugs)	5%	
Total	100%	/5

Critères de validation :

- Note globale $\geq 4/5$: Recette validée 
- Note $< 4/5$: Corrections nécessaires 

Signature recette :

Client : _____ Date : _____
Développeur : _____ Date : _____

13. Maintenance et évolutions

13.1 Support et maintenance

Période de garantie

****Durée :** 3 mois après livraison**

****Inclus :****

- Correction de bugs signalés
- Support par email (réponse $< 48h$)
- Mise à jour de sécurité critiques
- Ajustements mineurs UX

****Exclus :****

- Nouvelles fonctionnalités
- Formation additionnelle
- Modifications du cahier des charges

Niveaux de criticité

Niveau	Description	Délai de résolution
P0 - Critique	Application inaccessible	4 heures
P1 - Urgent	Fonctionnalité bloquante	24 heures
P2 - Important	Bug gênant mais contournable	72 heures
P3 - Mineur	Amélioration cosmétique	1 semaine

Processus de signalement

1. Client envoie email à : support@dev.com
2. Inclure :
 - Description du problème
 - Étapes pour reproduire
 - Captures d'écran si applicable
 - Navigateur et version
3. Développeur accuse réception < 4h
4. Investigation et correction selon criticité
5. Déploiement après validation
6. Notification au client

13.2 Évolutions prévues (Roadmap)

Phase 2 (Mois 3-4) - Automatisation

US-100 : Envoi automatique programmé

Description : Planifier l'envoi automatique des quittances chaque 1er du mois

Effort : 5 jours

Prérequis : Job scheduler (node-cron ou Bull)

Valeur métier : Gain de temps 100% sur les envois

US-101 : Gestion multi-utilisateurs

Description : Interface CRUD pour créer/gérer les utilisateurs

Effort : 3 jours

Prérequis : Table users avec rôles

Valeur métier : Délégation possible à plusieurs personnes

US-102 : Notifications

Description : Alertes email pour échéances, échecs, etc.

Effort : 2 jours

Prérequis : Service email configuré

Valeur métier : Proactivité sur les incidents

US-103 : Export comptable

Description : Export Excel/CSV des quittances pour le comptable

Effort : 3 jours

Prérequis : Librairie xlsx

Valeur métier : Simplification comptabilité

Phase 3 (Mois 5-6) - Fonctionnalités avancées

US-200 : Génération de factures

Description : Template de facturation pour prestations annexes

Effort : 5 jours

Prérequis : Nouveau modèle de données

Valeur métier : Centralisation de tous les documents

US-201 : Suivi des paiements

Description : Marquer quittances comme payées/impayées

Effort : 4 jours

Prérequis : Statut paiement en BDD

Valeur métier : Vision cash-flow en temps réel

US-202 : Relances automatiques

Description : Email de relance auto pour impayés après 7 jours

Effort : 3 jours

Prérequis : US-201 + Job scheduler

Valeur métier : Réduction des retards de paiement

US-203 : Signature électronique

Description : Signature numérique des PDFs (certificat)

Effort : 7 jours

Prérequis : Service de signature (DocuSign API ou équivalent)

Valeur métier : Valeur légale renforcée

Phase 4 (Mois 7-8) - Optimisations

US-300 : Migration S3

Description : Stocker PDFs sur AWS S3 au lieu de MySQL

Effort : 3 jours

Prérequis : Compte AWS, SDK configuré

Valeur métier : Réduction coûts stockage, meilleure performance

US-301 : Dashboard avancé

Description : Graphiques de revenus, statistiques détaillées

Effort : 5 jours

Prérequis : Librairie Chart.js ou Recharts

Valeur métier : Vision business améliorée

US-302 : API publique

Description : API REST documentée pour intégrations tierces

Effort : 7 jours

Prérequis : Documentation OpenAPI/Swagger

Valeur métier : Intégrations avec logiciels comptables

13.3 Maintenance préventive

Tâches mensuelles

- Mise à jour dépendances npm (patch versions)
- Analyse des logs d'erreurs (Sentry)
- Revue des performances (Lighthouse)
- Backup manuel BDD (vérification)
- Nettoyage des PDFs obsolètes (> 10 ans)
- Revue des comptes utilisateurs inactifs

Tâches trimestrielles

- Mise à jour dépendances npm (minor versions)
- Audit de sécurité (npm audit, Snyk)
- Tests de charge (k6 ou Artillery)
- Revue du code (dette technique)
- Mise à jour documentation
- Formation refresh pour le client

Tâches annuelles

- Mise à jour dépendances npm (major versions)
- Migration Next.js nouvelle version
- Audit de sécurité complet (pentest)
- Revue architecture (scalabilité)
- Renouvellement certificat SSL
- Archivage données > 10 ans

13.4 Documentation livrée

Documentation technique

1. README.md
 - Description projet
 - Installation locale
 - Configuration environnement

- Commandes principales
 - Architecture
2. ARCHITECTURE.md
 - Schéma architecture
 - Flux de données
 - Modèle BDD avec ERD
 - Décisions techniques
 3. API.md
 - Documentation de toutes les routes API
 - Exemples de requêtes/réponses
 - Codes d'erreur
 - Authentification
 4. DEPLOYMENT.md
 - Prérequis serveur
 - Étapes de déploiement
 - Configuration Nginx
 - Variables d'environnement
 - Gestion des certificats SSL
 - Procédure de rollback
 5. MAINTENANCE.md
 - Procédures de backup/restore
 - Monitoring et logs
 - Mise à jour dépendances
 - Troubleshooting commun

Documentation utilisateur

1. Guide Utilisateur (PDF)
 - Introduction
 - Première connexion
 - Créer un locataire
 - Générer une quittance
 - Consulter l'historique
 - Modifier les paramètres
 - FAQ
2. Vidéos tutorielles (optionnel)

- Parcours complet (5 min)
 - Génération de quittance (2 min)
 - Configuration entreprise (3 min)
3. Guide Administrateur (PDF)
- Gestion des utilisateurs
 - Gestion des propriétaires
 - Configuration avancée
 - Export comptable

14. Annexes

14.1 Glossaire

Terme	Définition
Quittance de loyer	Document attestant du paiement du loyer par le locataire
Avis de paiement	Document informant le propriétaire d'un versement à venir
RIB	Relevé d'Identité Bancaire (IBAN, BIC, codes bancaires)
SIRET	Numéro d'identification de l'établissement (14 chiffres)
Soft delete	Suppression logique (données conservées mais marquées inactives)
RGPD	Règlement Général sur la Protection des Données
MVP	Minimum Viable Product (version minimale fonctionnelle)
CRUD	Create, Read, Update, Delete (opérations de base)
JWT	JSON Web Token (format de token d'authentification)
ORM	Object-Relational Mapping (Prisma)
SSR	Server-Side Rendering (Next.js)
CSR	Client-Side Rendering

14.2 Références légales

Documents comptables

Code de commerce - Article L123-22

"Les documents comptables et les pièces justificatives sont conservés pendant dix ans."

LOI n° 2000-230 du 13 mars 2000 - Article 2

"L'écrit électronique a la même force probante que l'écrit sur support papier, sous réserve que puisse être dûment identifiée la personne dont il émane et qu'il soit établi et conservé dans des conditions de nature à en garantir l'intégrité."

Protection des données

RGPD - Article 5

Principes relatifs au traitement des données à caractère personnel :

- Licéité, loyauté, transparence
- Limitation des finalités
- Minimisation des données
- Exactitude
- Limitation de la conservation
- Intégrité et confidentialité

RGPD - Articles 15 à 22

Droits des personnes :

- Droit d'accès (Art. 15)
- Droit de rectification (Art. 16)
- Droit à l'effacement (Art. 17)
- Droit à la portabilité (Art. 20)

14.3 Contacts projet

Équipe développement

Lead Developer / Chef de projet

- Nom : [Votre nom]
- Email : dev@example.com
- Téléphone : +33 X XX XX XX XX
- Disponibilité : Lun-Ven 9h-18h

Client

Cosynight SAS

- Contact principal : Thomas VIRGILI
- Email : t.virgili@cosynight.fr
- Téléphone : 07.44.91.45.32
- Adresse : 10 rue de la bourse, 75002 Paris

Prestataires

Hébergement : Scaleway

- URL console : <https://console.scaleway.com>
- Support : support@scaleway.com

Email : Gmail (temporaire) → Resend (production)

- Support Resend : support@resend.com

14.4 Historique des versions du CDC

Version	Date	Auteur	Modifications
1.0	28/10/2025	Lead Dev	Création initiale complète
1.1	[Future]		Ajustements post-revue client

14.5 Acceptation du cahier des charges

Ce cahier des charges a été lu et approuvé par les parties suivantes :

Client - Cosynight SAS

Nom : Thomas VIRGILI

Signature : _____

Date : _____

Développeur

Nom : _____

Signature : _____

Date : _____

Ce document constitue la référence contractuelle pour le développement de la plateforme de gestion des quittances de loyer. Toute modification devra faire l'objet d'un avenant signé par les deux parties.

FIN DU CAHIER DES CHARGES

Document généré le : 28 octobre 2025

Nombre de pages : ~45

Durée de lecture : 2 heures

Durée de réalisation estimée : 8 semaines (2 mois)

Ce cahier des charges est un document évolutif. Il peut être amendé en fonction des retours du client et des contraintes techniques rencontrées pendant le développement.